Proiectarea și Analiza Algoritmilor

Trei probleme din lumea reală

Universitatea Politehnica Timișoara

Departamentul de Calculatoare și Tehnologia Informației

Anul II – Curs Aplicativ

Cuprins

1. Problema 1 – Rețeaua socială a unei petreceri
2. Problema 2 – Rețeaua de fibră optică din campus
3. Problema 3 – Livrări cu drone

Problema 1 – Enunț

Context

La o petrecere de bun-venit pentru studenți participă n persoane numerotate de la 1 la n. Înainte de petrecere, fiecare participant a comunicat organizatorilor lista perechilor de prieteni pe care îi cunoaște dinainte. Avem astfel o listă cu m perechi de tipul (a, b) care înseamnă că persoana a se cunoaște cu persoana b (relația este simetrică).

Obiectiv

Organizatorii vor să se asigure că nimeni nu rămâne izolat: o persoană poate fi prezentată unei alte persoane fie direct (dacă deja se cunosc), fie indirect, printr-un *lanț de prieteni comuni*. Mai mult, ei pregătesc un *tur de prezentare* pornind de la organizatorul principal, în care fiecare persoană este vizitată exact o singură dată.

Problema 1 – Cerințe

C1. Determinați câte *grupuri de cunoștințe* distincte există la petrecere (un grup este un set maximal de persoane în care fiecare poate ajunge la oricare alta prin lanț de prieteni).

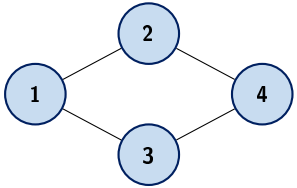
C2. Pentru fiecare grup, afișați persoanele care îl compun.

C3. Pornind de la organizatorul principal (persoana 1), generați ordinea de prezentare astfel încât *cele mai apropiate cunoștințe* (prieteni direcți, apoi prieteni-ai-prietenilor etc.) să fie prezentate primele.

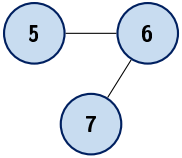
Date de intrare

`n`, `m`, urmat de `m` perechi (a_i, b_i).

Problema 1 – Exemplu vizual



Grup A: {1,2,3,4}



Grup B: {5,6,7}



Grup C: {8}

Rezultate așteptate: 3 grupuri; tur pornind din 1: $1, 2, 3, 4$.

C1 – Numărul de grupuri de cunoștințe

Cerința

Determinați câte componente disjuncte de cunoștințe există în rețeaua de participanți.

(Discuție la tablă: cum identificăm că două persoane fac parte din același grup? Ce structură folosim pentru a parcurge toate persoanele exact o singură dată?)

C1 – Idee de rezolvare

Idee: Iterăm prin toate persoanele. Când întâlnim o persoană încă nevizitată, pornim o *parcurgere* (BFS sau DFS) care marchează toți cei accesibili din ea. Fiecare astfel de inițiere corespunde unui grup nou.

Listing 1: Pseudocod (stil C)

```
nr_grupuri = 0
pentru i = 1, n:
    daca nu este vizitat[i]:
        nr_grupuri = nr_grupuri + 1
        BFS(i)          // marcheaza toti cei accesibili
afiseaza nr_grupuri
```

Complexitate: $O(n + m)$ – fiecare persoană și fiecare pereche sunt procesate o singură dată.

C1 – Implementare BFS în C

```
int vizitat[NMAX];
int coada[NMAX], st, dr;

void BFS(int sursa) {
    st = dr = 0;
    coada[dr++] = sursa;
    vizitat[sursa] = 1;
    while (st < dr) {
        int u = coada[st++];
        for (int i = 0; i < grad[u]; i++) {
            int v = vecin[u][i];
            if (!vizitat[v]) {
                vizitat[v] = 1;
                coada[dr++] = v;
            }
        }
    }
}

int numara_grupuri(int n) {
    int nr = 0;
    for (int i = 1; i <= n; i++)
        if (!vizitat[i]) { nr++; BFS(i); }
    return nr;
}
```

C2 – Membrii fiecărui grup

Cerința

Pentru fiecare grup de cunoștințe, afișați lista persoanelor care îl compun.

(Discuție: cum extindem BFS-ul anterior pentru a colecta și a afișa persoanele găsite, nu doar pentru a le număra?)

C2 – Idee de rezolvare

Idee: Reluăm bucla din C1, dar de fiecare dată când inițiem o parcurgere nouă, colectăm într-un vector toate persoanele vizitate *pe parcursul acelei parcurgeri*.

Listing 2: Pseudocod

```
nr_grupuri = 0
pentru i = 1, n:
    daca nu este vizitat[i]:
        nr_grupuri = nr_grupuri + 1
        membri = lista_vida()
        BFS_colecteaza(i, membri)
        afiseaza "Grup", nr_grupuri, ":", membri
```

Fiecare persoană apare în exact un grup, deci complexitatea rămâne $O(n+m)$.

C2 – Implementare în C

```c
void BFS_colecteaza(int sursa, int *membri, int *nr_membri) {
    int coada[NMAX], st = 0, dr = 0;
    coada[dr++] = sursa;
    vizitat[sursa] = 1;
    *nr_membri = 0;
    while (st < dr) {
        int u = coada[st++];
        membri[(*nr_membri)++] = u;
        for (int i = 0; i < grad[u]; i++) {
            int v = vecin[u][i];
            if (!vizitat[v]) {
                vizitat[v] = 1;
                coada[dr++] = v;
            }
        }
    }
}

void afiseaza_grupuri(int n) {
    int membri[NMAX], nr, grup = 0;
    for (int i = 1; i <= n; i++)
        if (!vizitat[i]) {
            BFS_colecteaza(i, membri, &nr);
            printf("Grup %d:", ++grup);
            for (int k = 0; k < nr; k++) printf(" %d", membri[k]);
            printf("\n");
        }
}
```

C3 – Ordinea de prezentare

Cerința

Pornind din persoana 1 (organizatorul), generați ordinea de prezentare astfel încât prietenii direcți să fie vizitați primii, apoi prietenii-ai-prietenilor, ș.a.m.d.

(Discuție: ce diferență face faptul că vrem să respectăm distanța în „pași de cunoaștere"? Ce tip de parcurgere garantează acest lucru?)

C3 – Idee de rezolvare

Observație cheie: BFS-ul vizitează persoanele în ordinea crescătoare a distanței (numărul de pași) față de sursă. Exact ce ne trebuie.

Listing 3: Pseudocod

```
BFS(1):
    coada = [1]; vizitat[1] = 1; nivel[1] = 0
    cat timp coada nu e vida:
        u = scoate_din_coada()
        afiseaza u, nivel[u]
        pentru fiecare prieten v al lui u:
            daca nu este vizitat[v]:
                vizitat[v] = 1
                nivel[v] = nivel[u] + 1
                pune_in_coada(v)
```

Garanție: ordinea de scoatere din coadă este și ordinea de prezentare cerută.

C3 – Implementare în C

```c
int nivel[NMAX];

void tur_prezentare(int sursa) {
    int coada[NMAX], st = 0, dr = 0;
    for (int i = 0; i < NMAX; i++) { vizitat[i] = 0; nivel[i] = -1; }

    coada[dr++] = sursa;
    vizitat[sursa] = 1;
    nivel[sursa] = 0;

    printf("Ordine de prezentare pornind din %d:\n", sursa);
    while (st < dr) {
        int u = coada[st++];
        printf("  pas %d -> persoana %d\n", nivel[u], u);
        for (int i = 0; i < grad[u]; i++) {
            int v = vecin[u][i];
            if (!vizitat[v]) {
                vizitat[v] = 1;
                nivel[v] = nivel[u] + 1;
                coada[dr++] = v;
            }
        }
    }
}
```

Problema 2 – Enunț

Context

Administrația campusului dorește să conecteze prin fibră optică toate cele n clădiri ale campusului. Compania de telecomunicații a evaluat toate variantele posibile de cabluri directe între perechi de clădiri și a comunicat o listă cu m trasee posibile, fiecare cu un cost în euro: (u_i, v_i, c_i) – traseu între clădirea u_i și clădirea v_i la prețul c_i.

Obiectiv

Se dorește alegerea unui *subansamblu de trasee* astfel încât oricare două clădiri să fie conectate (direct sau prin alte clădiri intermediare), iar costul total al cablării să fie *minim*.

Problema 2 – Cerințe

C1. Calculați costul minim total al rețelei de fibră.

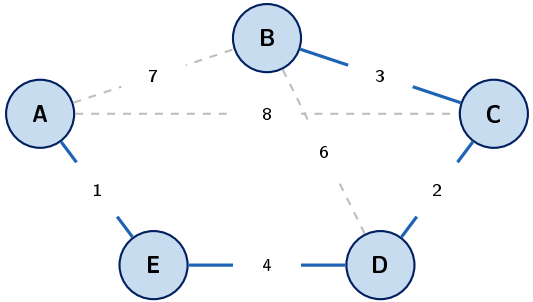
C2. Afișați lista efectivă a traseelor care vor fi cablate.

C3. Verificați dacă rețeaua este *realizabilă* (toate clădirile pot fi conectate cu traseele oferite de companie); dacă nu, afișați câte trasee suplimentare ar fi necesare *cel puțin*.

Date de intrare

`n`, `m`, urmat de `m` triplete (u_i, v_i, c_i).

Problema 2 – Exemplu vizual



trasee alese

trasee neutilizate

Cost total: $1 + 4 + 2 + 3 = 10$

C1 – Costul minim al rețelei

Cerința

Calculați costul minim total pentru cablarea rețelei astfel încât toate clădirile să fie conectate.

(Discuție: avem două abordări clasice – una care „crește un arbore" pornind dintr-o clădire (Prim), alta care „adună trasee în ordinea costului" (Kruskal). Care e mai potrivită când avem multe trasee posibile?)

C1 – Algoritmul Kruskal

Idee: Sortăm toate traseele crescător după cost. Le parcurgem în ordine; adăugăm un traseu doar dacă *nu creează un ciclu* (adică nu unește două clădiri deja conectate). Folosim structura `Union-Find` pentru verificare rapidă.

Listing 4: Pseudocod Kruskal

```
sorteaza traseele crescator dupa cost
init Union-Find cu n clase
cost_total = 0; nr_alese = 0
pentru fiecare traseu (u, v, c) in ordine:
    daca find(u) != find(v):
        union(u, v)
        cost_total += c
        nr_alese++
        daca nr_alese == n - 1: oprire
afiseaza cost_total
```

Complexitate: $O(m \log m)$ – domină sortarea.

C1 – Implementare Kruskal (1/2)

```
typedef struct { int u, v, c; } Traseu;

int parinte[NMAX], rang[NMAX];

void init_uf(int n) {
    for (int i = 1; i <= n; i++) { parinte[i] = i; rang[i] = 0; }
}

int find(int x) {
    while (parinte[x] != x) {
        parinte[x] = parinte[parinte[x]];  // compresie
        x = parinte[x];
    }
    return x;
}

int union_uf(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return 0;
    if (rang[ra] < rang[rb]) parinte[ra] = rb;
    else if (rang[ra] > rang[rb]) parinte[rb] = ra;
    else { parinte[rb] = ra; rang[ra]++; }
    return 1;
}
```

C1 – Implementare Kruskal (2/2)

```
int cmp(const void *a, const void *b) {
    return ((Traseu*)a)->c - ((Traseu*)b)->c;
}

long kruskal(Traseu *T, int m, int n) {
    qsort(T, m, sizeof(Traseu), cmp);
    init_uf(n);
    long cost_total = 0;
    int nr_alese = 0;
    for (int i = 0; i < m && nr_alese < n - 1; i++) {
        if (union_uf(T[i].u, T[i].v)) {
            cost_total += T[i].c;
            nr_alese++;
        }
    }
    return cost_total;
}
```

C2 – Lista traseelor alese

Cerința

Pe lângă costul total, afișați și *care* trasee au fost selectate pentru cablare.

(Discuție: ce mică modificare aducem algoritmului anterior pentru a memora și raporta efectiv traseele selectate, nu doar suma costurilor?)

C2 – Implementare cu reținerea traseelor

Idee: Reținem într-un vector traseele acceptate de `union_uf`.

```
long kruskal_cu_lista(Traseu *T, int m, int n,
                      Traseu *alese, int *nr_alese_out) {
    qsort(T, m, sizeof(Traseu), cmp);
    init_uf(n);
    long cost_total = 0;
    int nr = 0;
    for (int i = 0; i < m && nr < n - 1; i++) {
        if (union_uf(T[i].u, T[i].v)) {
            alese[nr++] = T[i];
            cost_total += T[i].c;
        }
    }
    *nr_alese_out = nr;
    return cost_total;
}

void afiseaza_trasee(Traseu *alese, int nr) {
    printf("Trasee alese:\n");
    for (int i = 0; i < nr; i++)
        printf("  %d -- %d  (cost %d)\n",
               alese[i].u, alese[i].v, alese[i].c);
}
```

C3 – Verificarea realizabilității

Cerința

Stabiliți dacă rețeaua poate fi efectiv construită cu traseele oferite. Dacă nu, raportați câte trasee *suplimentare* ar trebui adăugate cel puțin.

(Discuție: la finalul algoritmului Kruskal, ce înseamnă dacă am ales mai puțin de $n-1$ trasee? Câte componente independente rămân?)

C3 – Idee și implementare

Observație: Dacă Kruskal selectează doar $k < n-1$ trasee, înseamnă că am rămas cu $n-k$ componente disjuncte. Pentru a le uni este nevoie de exact $(n-k)-1$ trasee suplimentare.

```
void verifica_realizabilitate(Traseu *T, int m, int n) {
    qsort(T, m, sizeof(Traseu), cmp);
    init_uf(n);
    int nr_alese = 0;
    long cost_total = 0;
    for (int i = 0; i < m && nr_alese < n - 1; i++)
        if (union_uf(T[i].u, T[i].v)) {
            cost_total += T[i].c;
            nr_alese++;
        }

    if (nr_alese == n - 1) {
        printf("Realizabila. Cost = %ld\n", cost_total);
    } else {
        int componente = n - nr_alese;
        printf("Nerealizabila. Componente ramase: %d\n", componente);
        printf("Trasee suplimentare necesare: %d\n", componente - 1);
    }
}
```

Problema 3 – Enunț

Context

O companie de livrări operează cu drone într-un oraș. Există n *puncte de tranzit* numerotate $1 \dots n$ (depozitul central este punctul 1). Între unele puncte există rute aeriene autorizate; fiecare rută este o pereche (u, v, t), unde t este timpul în minute necesar pentru a parcurge ruta cu o dronă (rutele sunt bidirecționale, timpii sunt strict pozitivi).

Obiectiv

Pentru a planifica livrările, compania are nevoie să cunoască *timpul minim* de zbor între depozit și orice alt punct de tranzit, precum și traseul efectiv urmat de fiecare dronă.

Problema 3 – Cerințe

C1. Calculați timpul minim de zbor de la depozit (1) către fiecare punct de tranzit.

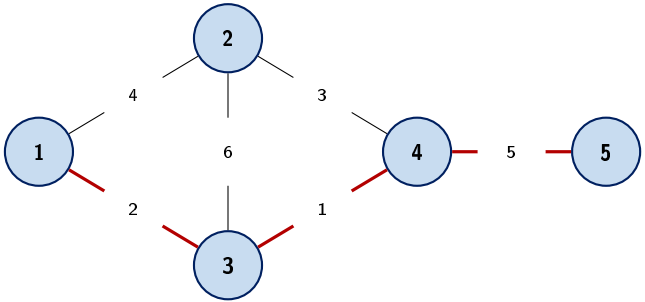
C2. Pentru un punct destinație d dat, reconstruiți *traseul efectiv* pe care îl va urma drona.

C3. Identificați *cea mai îndepărtată* destinație (în timp) de la depozit – vom planifica pentru ea o dronă cu autonomie suplimentară.

Date de intrare

n, m, urmat de m triplete (u_i, v_i, t_i) cu $t_i > 0$.

Problema 3 – Exemplu vizual



Timpi minimi din 1:
$t(1) = 0$, $t(2) = 4$,
$t(3) = 2$, $t(4) = 3$,
$t(5) = 8$

Traseul către 5:
$1 \to 3 \to 4 \to 5$

C1 – Timpii minimi de la depozit

Cerința

Calculați timpul minim de zbor de la punctul 1 către fiecare alt punct $i \in \{2, \ldots, n\}$.

(Discuție: de ce un simplu BFS nu funcționează aici? Ce face diferența între „număr de rute" și „timp total"? Care e algoritmul potrivit pentru ponderi pozitive?)

C1 – Algoritmul Dijkstra (idee)

Idee: Menținem un vector `dist[]` cu timpul minim cunoscut până la fiecare punct. La fiecare pas extragem punctul cu cel mai mic timp *neprocesat încă* și actualizăm (*relaxăm*) timpii vecinilor săi. Pentru extragerea minimului folosim o coadă cu priorități (min-heap).

Listing 5: Pseudocod Dijkstra

```
1  init dist[i] = +inf pentru toti i; dist[sursa] = 0
2  insereaza (0, sursa) in heap
3  cat timp heap nu e gol:
4      (d, u) = extrage_min(heap)
5      daca d > dist[u]: continua  // intrare invechita
6      pentru fiecare ruta (u, v, t):
7          daca dist[u] + t < dist[v]:
8              dist[v] = dist[u] + t
9              parinte[v] = u
0              insereaza (dist[v], v) in heap
```

Complexitate: $O((n+m)\log n)$ cu min-heap.

C1 – Implementare Dijkstra (1/2)

```
typedef struct { int nod; long dist; } Heap;

Heap H[NMAX];
int sz = 0;

void hpush(int nod, long dist) {
    H[sz].nod = nod; H[sz].dist = dist;
    int i = sz++;
    while (i > 0) {
        int p = (i - 1) / 2;
        if (H[p].dist <= H[i].dist) break;
        Heap t = H[p]; H[p] = H[i]; H[i] = t;
        i = p;
    }
}

Heap hpop(void) {
    Heap top = H[0];
    H[0] = H[--sz];
    int i = 0;
    for (;;) {
        int l = 2*i+1, r = 2*i+2, m = i;
        if (l < sz && H[l].dist < H[m].dist) m = l;
        if (r < sz && H[r].dist < H[m].dist) m = r;
        if (m == i) break;
        Heap t = H[m]; H[m] = H[i]; H[i] = t;
        i = m;
    }
    return top;
}
```

C1 – Implementare Dijkstra (2/2)

```
long dist[NMAX];
int parinte[NMAX];
// vecin[u][k] = nod adiacent;  cost[u][k] = timp ruta
// grad[u]     = numar de vecini

void dijkstra(int sursa, int n) {
    for (int i = 1; i <= n; i++) {
        dist[i] = LONG_MAX;
        parinte[i] = -1;
    }
    dist[sursa] = 0;
    sz = 0;
    hpush(sursa, 0);

    while (sz > 0) {
        Heap top = hpop();
        int u = top.nod;
        if (top.dist > dist[u]) continue;
        for (int k = 0; k < grad[u]; k++) {
            int v = vecin[u][k];
            long nd = dist[u] + cost[u][k];
            if (nd < dist[v]) {
                dist[v] = nd;
                parinte[v] = u;
                hpush(v, nd);
            }
        }
    }
}
```

C2 – Reconstruirea traseului

Cerința

Pentru un punct destinație d, afișați secvența punctelor pe care drona le parcurge: $1 \to \ldots \to d$.

(Discuție: în timpul Dijkstra am reținut, pentru fiecare punct, „cine este predecesorul" pe drumul minim. Cum reconstruim traseul folosind acest vector?)

C2 – Reconstruirea traseului (implementare)

Idee: Pornim din d și urcăm prin `parinte[]` până ajungem la sursă, apoi inversăm ordinea.

```
void afiseaza_traseu(int destinatie) {
    if (dist[destinatie] == LONG_MAX) {
        printf("Destinatia %d nu este accesibila.\n", destinatie);
        return;
    }
    int traseu[NMAX], len = 0;
    for (int v = destinatie; v != -1; v = parinte[v])
        traseu[len++] = v;

    printf("Traseu (timp total %ld): ", dist[destinatie]);
    for (int i = len - 1; i >= 0; i--) {
        printf("%d", traseu[i]);
        if (i > 0) printf(" -> ");
    }
    printf("\n");
}
```

Complexitate: $O(n)$ pentru reconstrucție.

C3 – Cea mai îndepărtată destinație

Cerința

Determinați punctul d^* pentru care timpul minim de zbor de la depozit este cel mai mare (excluzând punctele inaccesibile).

(Discuție: după rularea Dijkstra, ce trebuie să mai facem? Cum tratăm punctele cu `dist` infinit?)

C3 – Implementare

Idee: O simplă trecere prin vectorul `dist[]` – păstrăm maximul printre valorile finite.

```
void cea_mai_indepartata(int n) {
    int d_star = -1;
    long max_t = -1;
    for (int i = 1; i <= n; i++) {
        if (dist[i] != LONG_MAX && dist[i] > max_t) {
            max_t = dist[i];
            d_star = i;
        }
    }
    if (d_star == -1) {
        printf("Nicio destinatie accesibila.\n");
    } else {
        printf("Cea mai indepartata: punctul %d, timp = %ld\n",
               d_star, max_t);
        afiseaza_traseu(d_star);
    }
}
```

Complexitate: $O(n)$.

Sinteză – ce am exersat astăzi

- **Problema 1 –** *Parcurgere și conexitate*
  - BFS / DFS, componente conexe, ordine în pași de distanță.
  - Complexitate $O(n + m)$.
- **Problema 2 –** *Arbore parțial de cost minim*
  - Kruskal cu Union-Find (compresie + rang).
  - Complexitate $O(m \log m)$.
- **Problema 3 –** *Drumuri minime cu ponderi pozitive*
  - Dijkstra cu min-heap, reconstrucție drum prin predecesori.
  - Complexitate $O((n + m) \log n)$.

De reținut

Aceeași reprezentare a datelor (liste de adiacență) și aceleași tehnici se aplică la o gamă largă de probleme practice – rețele sociale, infrastructură, logistică, rutare, etc.

Mulțumesc!

Întrebări?

Universitatea Politehnica Timișoara
Departamentul de Calculatoare și Tehnologia Informației